Talos Debt Follow-up: Database Summary

Summary of `talos_schema.sql` (PostgreSQL 15+ with pgvector, hosted in a Saudi region).

In short: the file builds the whole database in one run. It creates the tables, links them so data can't get mixed up, makes the database enforce the key business rules itself, and controls who can see what.

---

1. Setup

Turns on three add-ons:

- ID generation
- case-insensitive emails
- pgvector for AI search

Creates three schemas:

- `app` : business data
- `ai` : AI retrieval and usage
- `audit` : activity log

2. Helpers

- Small functions that read which client and which user the current request is for.
- A function that stamps `updated_at` when a row changes.

3. Shared tables

Data that belongs to Talos, not to a single client:

Table	Purpose
`instrument_types`	The Najiz list of enforceable instruments
`sender_numbers`	The WhatsApp numbers you send from
`webhook_events`	Raw updates from WhatsApp and the payment gateway

4. Clients and settings

Table	Purpose
`clients`	Company details, IBAN, plan
`users`	Owner, admin, staff, viewer
`follow_up_rules`	How often to message, working hours, when to escalate

5. Debtors and debts

Table	Purpose
`debtors`	One person or company
`debtor_phones`	Each phone number, with its own status
`instruments`	Promissory notes, cheques and so on
`debts`	Amount, reason, status
`documents`	Details of each uploaded file
`imports`	Excel uploads, with the errors found in each row

6. Messaging

Table	Purpose
`message_templates`	Approved WhatsApp texts
`messages`	Every message in and out (also the proof of contact)
`follow_up_tasks`	The scheduler's to-do list
`promises`	Promises to pay
`disputes`	Debtors saying the amount is wrong

7. Payments

Table	Purpose
`payment_links`	Links that pay into the client's own account
`payments`	Payments; one only counts once the client approves it

8. Legal

`legal_cases` tracks each filing pack from "ready" to "closed".

- It can't be submitted without a person's approval.
- Each debt can have only one open case.

9. Notifications

`notifications` holds alerts for the client's users: new replies, payments to approve, disputes.

10. AI tables

Table	Purpose
`ai.document_chunks`	Document text and its search vectors, so the agent can find the right facts
`ai.agent_runs`	Each AI call, with its tokens and cost

11. Activity log

- `audit.activity_log` records every change to the key tables automatically.
- Nobody can edit or delete these records, not even the database owner.

12. Business rules (triggers)

- **Payment approved:** the balance updates. At zero, the debt becomes "paid" and its reminders are cancelled.
- **Dispute opened:** the debt is paused and its reminders are cancelled.

13. Dashboard views

View	Shows
`v_debt_overview`	One line per debt
`v_client_totals`	Total owed, recovered, promised, disputed, in legal, enforceable share, paper notes at risk

14. Client separation

- Every query only sees the current client's rows.
- If no client is set, it sees nothing.
- The back end must run `SET LOCAL app.client_id = '<client id>';` on every request.

15. Roles

Role	Used by	Access
`talos_app`	The web app	Only sees one client at a time
`talos_worker`	The scheduler and webhooks	Can see all clients (keep this code small and reviewed)

16. Starting data

Fills in 18 instrument types:

- the 16 types from the Najiz screenshots
- "plain contract" and "invoice", which need a court case before execution

---

To agree with Sal

- **Setting the client:** every request must set `app.client_id` before it runs any query.
- **Embedding size:** the vector column is set to 1,536 numbers per text piece. Change it to match the embedding model Sal picks, before loading any data.